COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 18, 2026	**DATE SUBMITTED:**	October 27, 2026
NAME:	Dela Cruz, Jared Isaiah P.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

Hands-on Activity 10.1 **Graphs**

1. PROCEDURE OUTPUT

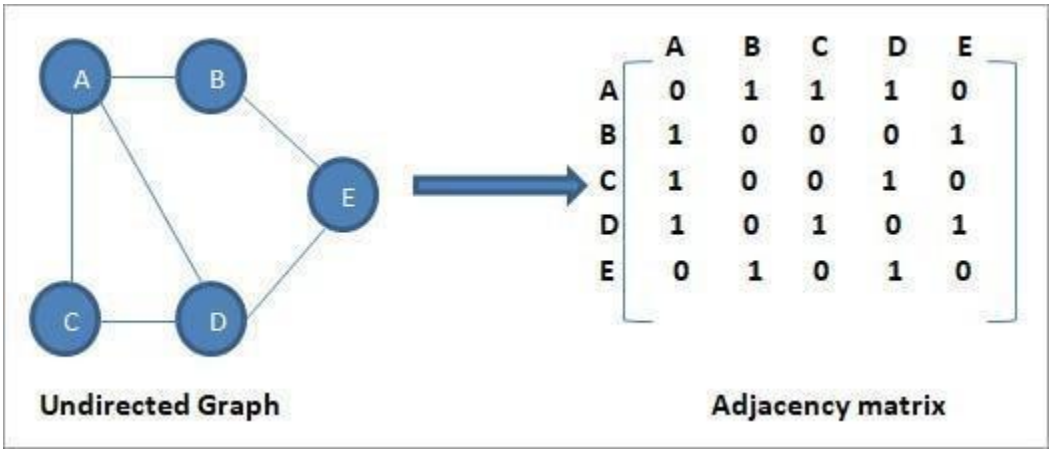
ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

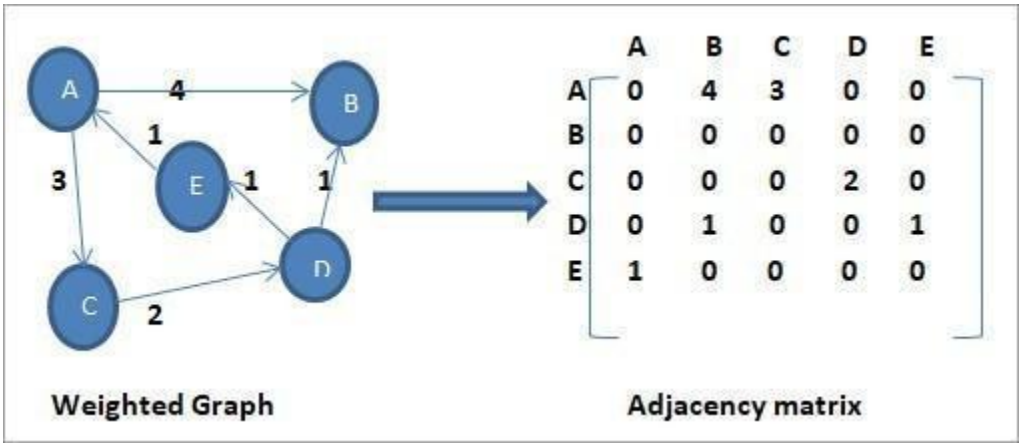
Following are the basic operations that we can perform on the graph data structure:

- **Add a vertex:** Adds vertex to the graph.
- **Add an edge:** Adds an edge between the two vertices of a graph.
- **Display the graph vertices:** Display the vertices of a graph.

A.2. Adjacency Matrix

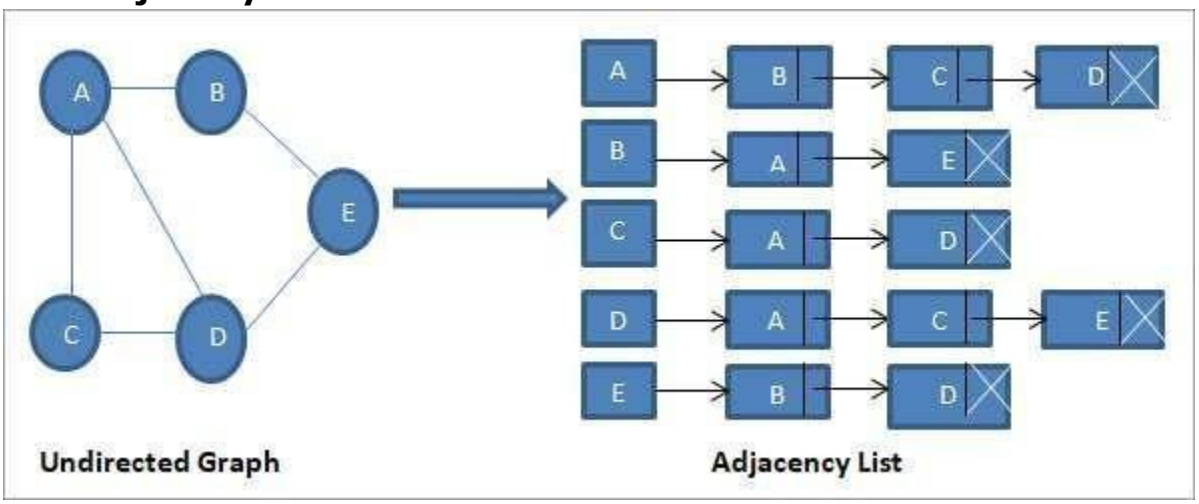


	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	0	1
C	1	0	0	1	0
D	1	0	1	0	1
E	0	1	0	1	0

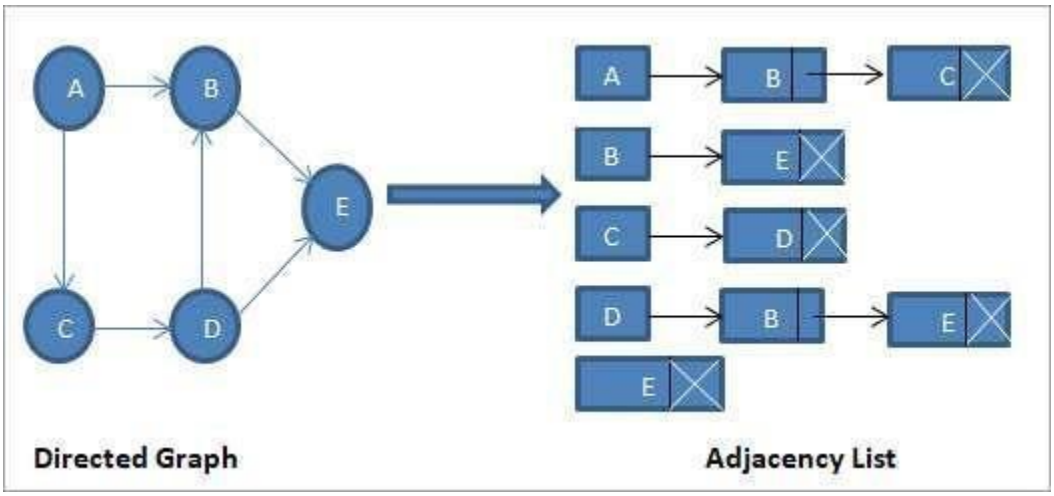


	A	B	C	D	E
A	0	4	3	0	0
B	0	0	0	0	0
C	0	0	0	2	0
D	0	1	0	0	1
E	1	0	0	0	0

A.3. Adjacency List



A.4. Adjacency List



Sample Code:

```
#include <iostream>

// stores adjacency
list items struct
adjNode {
    int
    val,
    cost;
    adjNode
    * next;
};
// structure to
store edges
struct graphEdge
{
    int start_ver, end_ver, weight;
};
class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight,
        adjNode* head)

        { adjNode* newNode = new adjNode;
        newNode->v
        al =
        value;
        newNode->
        c ost =
        weight;
        newNode->next = head;    // point new node to current head

        return newNode;
    }
    int N; // number of nodes in the
graph public:
    adjNode **head;                 //adjacency list as array of
    pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new
        adjNode*[N](
        ); this->N =
        N;
        // initialize head pointer for
        all vertices for (int i = 0; i
        < N; ++i)
            head[i] = nullptr;
        // construct directed graph by adding
        edges to it for (unsigned i = 0; i <
        n; i++) {
```

```
            int start_ver =
            edges[i].start_ver; int
            end_ver =
            edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight,
            head[start_ver]);

            // point head pointer to
            new node head[start_ver] =
            newNode;
        }
    }
    // Destructor
    ~DiaGraph() {
        for (int i = 0;
            i < N; i++)
            delete[]
            head[i];
        delete[] head;
    }
};
// print all adjacent vertices of
given vertex void
display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
             << ", " <<
        ptr->cost << ") "; ptr
        = ptr->next;
    }
    std::cout << std::endl;
}
// graph
implementat
ion int
main()
{
    // graph
    edges array.
    graphEdge
    edges[] = {
            // (x, y, w) -> edge from x to y with weight w
            {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6;       // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph std::cout<<"Graph
    adjacency list "<<std::endl<<"(start_vertex, end_vertex,
```

```
weight):"<<std::endl;
    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of
        vertex i
        display_AdjList(diagraph.head[i
        ], i);
    }
    return 0;
}
```

Implement the given code and indicate your output as a table in section 6.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Step 1. Include the required header files, as follows:

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T> class Graph;
```

Step 2. Write the following struct in order to implement an edge in our graph:

```
template <typename T> struct Edge
{
size_t src; size_t dest; T weight;
// To compare edges, only compare their weights,
// and not the source/destination vertices inline bool operator<(const
Edge<T> &e) const
{
return this->weight < e.weight;
}
inline bool operator>(const Edge<T> &e) const
{
return this->weight > e.weight;
}
};
```

Step 3. Next, overload the << operator for the graph so that it can be printed out using the following function:

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges =
        G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " <<
        e.weight << "}, "; os << std::endl;
    }
    return os;
}
```

Step 4. Implement the graph data structure that uses an edge list representation as follows:

```
template
<typename
T> class
Graph
{
public:
    // Initialize the graph with N
    vertices Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in
    the graph auto vertices() const
    {
        return V;
    }
    // Return all edges in
    the graph auto &edges()
    const
    {
        return edge_list;
    }

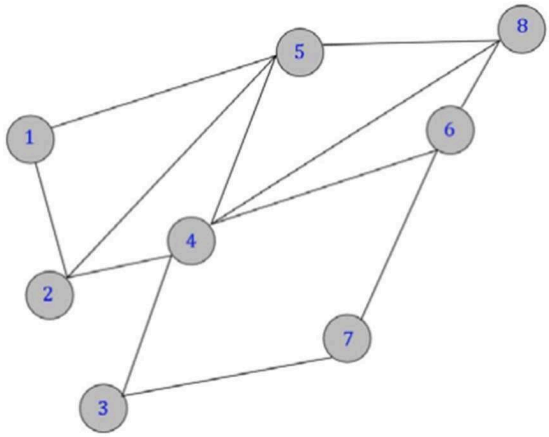
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are
        within range if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges
    from vertex v auto
    outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>>
```

```
            edges_from_v; for
            (auto &e : edge_list)
            {
                if (e.src == v)
                    edges_from_v.emplace_ba
                    ck(e);
            }
            return edges_from_v;
        }
        // Overloads the << operator so a graph be written directly to a
        stream
        // Can be used as std::cout << obj <<
        std::endl; template <typename U>
        friend std::ostream &operator<<(std::ostream &os, const Graph<U>
        &G);private:
        size_t V; // Stores number of
        vertices in graph
        std::vector<Edge<T>> edge_list;
};
```

Step 5. Now, we need a function to perform DFS for our graph. Implement it as follows:

```
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from
    vertex ID 1 while (!stack.empty())
    {
        auto current_vertex =
        stack.top(); stack.pop();
        // If the current vertex hasn't been visited
        in the past if (visited.find(current_vertex)
        == visited.end())
        {
            visited.insert(current_vert
            ex);
            visit_order.push_back(curre
            nt_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the
                stack. if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

Step 6. We shall test our implementation of the DFS on the graph shown here:



Use the following function to create and return the graph:

```
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t,
    T>>> edges; edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first,
    j.second}); return G;
}
```

Note the use of null values for edge weights since DFS does not require edge weights. A simpler implementation of the graph could have omitted the edge weights entirely without affecting the behavior of our DFS algorithm.

Step 7. Finally, add the following test and driver code, which runs our DFS implementation and prints the output:

```
template <typename T> void test_DFS()
{
// Create an instance of and print the graph auto
G = create_reference_graph<unsigned>();
```

```
std::cout << G << std::endl;
// Run DFS starting from vertex ID 1 and print the order
// in which vertices are visited.
std::cout << "DFS Order of vertices: " << std::endl;
auto dfs_visit_order = depth_first_search(G, 1);
for (auto v : dfs_visit_order) std::cout << v << std::endl;
}
int main()
{
using T = unsigned; test_DFS<T>(); return 0;
}
```

Step 8. Compile and run the preceding code. **Include your output as a table in section 6.B.2. Breadth-First Search**

Step 1: Include the required header files and declare the graph as follows:

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T> class Graph;
```

Step 2: Write the following struct, which represents an edge in our graph:

```
template <typename T> struct Edge
{
size_t src; size_t dest; T weight;

inline bool operator<(const Edge<T> &e) const
{
return this->weight < e.weight;
}

inline bool operator>(const Edge<T> &e) const
{
return this->weight > e.weight;
}
};
```

Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:

```
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
```

```
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges =
        G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }
}
```

Step 4: Write a class to define our graph data structure, as shown here:

```
template
<typename
T> class
Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 &&
            e.src <= V &&
            e.dest >= 1 &&
            e.dest <= V)
            edge_list.empl
            ace_back(e);
                            elsestd::cerr << "Vertex out of bounds" <<
                                std::endl;
    }



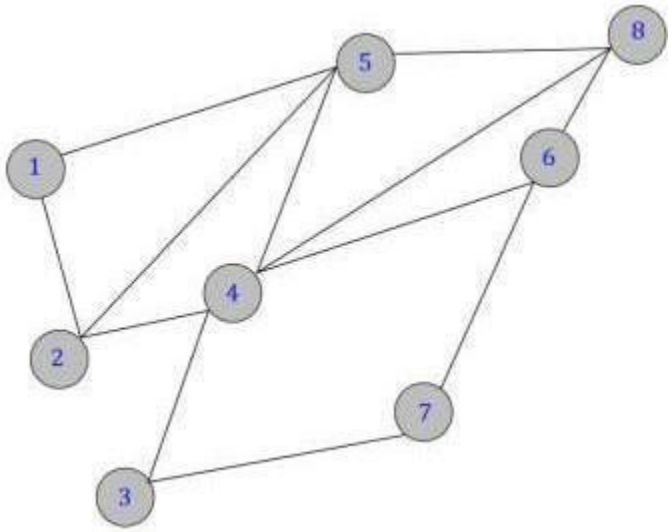
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>>
        edges_from_v; for
        (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
```

```
                }
            }
            return edges_from_v;
        }

        template <typename T>
        friend std::ostream &operator<<(std::ostream &os, const Graph<T>
        &G);

    private:
        size_t V;
        std::vector<Edge<T>> edge_list;
    };
```

Step 5: For this exercise, we shall test our implementation of BFS on the following graph:



We need a function to create and return the required graph. Note that while edge weights are assigned to each edge in the graph, this is not necessary since the BFS algorithm does not need to use edge weights. Implement the function as follows:

```
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t,
    T>>> edges; edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
```

```
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

Step 6: Implement the breadth-first search like so:

```
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from
    vertex ID 1 while (!queue.empty())
    {
        auto current_vertex =
        queue.front(); queue.pop();
        // If the current vertex hasn't been visited
        in the past if (visited.find(current_vertex)
        == visited.end())
        {
            visited.insert(current_vert
            ex);
            visit_order.push_back(curre
            nt_vertex);
            for (auto e :
                G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

Step 7: Add the following test and driver code that creates the reference graph, runs BFS starting from vertex 1, and outputs the results:

```
template <typename T> void test_BFS()
{
// Create an instance of and print the graph auto
G = create_reference_graph<unsigned>(); std::cout
<< G << std::endl;
// Run BFS starting from vertex ID 1 and print the order
// in which vertices are visited.
std::cout << "BFS Order of vertices: " << std::endl;
auto bfs_visit_order = breadth_first_search(G, 1);
```

```
for (auto v : bfs_visit_order)
std::cout << v << std::endl;
}

int main()
{
using T = unsigned; test_BFS<T>(); return 0;
}
```

Include the out and observation as a table in section 6.

Code Snippets	

```cpp
#include <iostream>
#include <map>
#include <vector>

class MatrixGraph{
private:
    std::vector<char> vertices;
    std::vector<std::vector<int>> matrix;
    std::vector<std::vector<int>> weightedMatrix;

public:
    void addVertex(char v){
        vertices.push_back(v);

        for (auto& row : matrix)
            row.push_back(0);

        matrix.push_back(std::vector<int>(vertices.size(), 0));

        for (auto& row : weightedMatrix)
            row.push_back(0);

        weightedMatrix.push_back(std::vector<int>(vertices.size(), 0));
    }
```

```cpp
    void addEdge(char u, char v){
        int i = findVertex(u);
        int j = findVertex(v);

        if (i != -1 && j != -1)
        {
            matrix[i][j] = 1;
            matrix[j][i] = 1;
        }
    }
```

```
void addWeightedEdge(
    char u,
    char v,
    int weight
){
    int i = findVertex(u);
    int j = findVertex(v);

    if (i != -1 && j != -1)
        weightedMatrix[i][j] = weight;
}
```

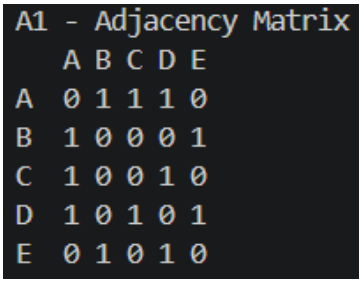
```
int main()
{

    MatrixGraph graph;

    graph.addVertex('A');
    graph.addVertex('B');
    graph.addVertex('C');
    graph.addVertex('D');
    graph.addVertex('E');
    graph.addEdge('A', 'B');
    graph.addEdge('B', 'E');
    graph.addEdge('A', 'D');
    graph.addEdge('A', 'C');
    graph.addEdge('D', 'E');
    graph.addEdge('C', 'D');

    std::cout << "A2 - Adjacency Matrix\n";

    graph.displayMatrix();
```

Output	
Observation	An undirected adjacency matrix uses a grid of rows and columns to map node connections, marking a 1 for direct links and a 0 for unconnected pairs. Because pathways travel both ways, the matrix mirrors itself perfectly across its main diagonal.

A.2: Undirected Adjacency Matrix

Code Snippets

```
class MatrixGraph{
private:
    std::vector<char> vertices;
    std::vector<std::vector<int>> matrix;
    std::vector<std::vector<int>> weightedMatrix;

public:
    void addVertex(char v){
        vertices.push_back(v);

        for (auto& row : matrix)
            row.push_back(0);

        matrix.push_back(std::vector<int>(vertices.size(), 0));

        for (auto& row : weightedMatrix)
            row.push_back(0);

        weightedMatrix.push_back(std::vector<int>(vertices.size(), 0));
    }

    void addEdge(char u, char v){
        int i = findVertex(u);
        int j = findVertex(v);

        if (i != -1 && j != -1)
        {
            matrix[i][j] = 1;
            matrix[j][i] = 1;
        }
    }
```

```cpp
void addWeightedEdge(
    char u,
    char v,
    int weight
){
    int i = findVertex(u);
    int j = findVertex(v);

    if (i != -1 && j != -1)
        weightedMatrix[i][j] = weight;
}

void displayMatrix(){
    std::cout << "   ";

    for (char v : vertices)
        std::cout << v << " ";

    std::cout << std::endl;

    for (size_t i = 0; i < vertices.size(); ++i){
        std::cout << vertices[i] << "  ";

        for (size_t j = 0; j < vertices.size(); ++j)
            std::cout << matrix[i][j] << " ";

        std::cout << std::endl;
    }
}
```

```
void displayWeightedMatrix(){
    std::cout << "    ";

    for (char v : vertices)
        std::cout << v << " ";

    std::cout << std::endl;

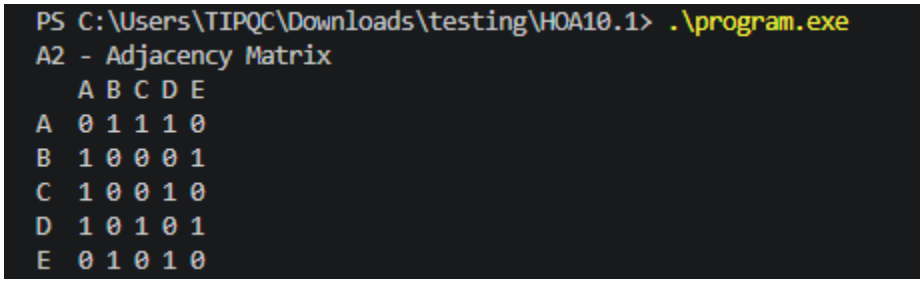
    for (size_t i = 0; i < vertices.size(); ++i){
        std::cout << vertices[i] << "   ";

        for (size_t j = 0; j < vertices.size(); ++j)
            std::cout << weightedMatrix[i][j] << " ";

        std::cout << std::endl;
    }
}
```

```
MatrixGraph weightedGraph;

weightedGraph.addVertex('A');
weightedGraph.addVertex('B');
weightedGraph.addVertex('C');
weightedGraph.addVertex('D');
weightedGraph.addVertex('E');
weightedGraph.addWeightedEdge('A', 'B', 4);
weightedGraph.addWeightedEdge('A', 'C', 3);
weightedGraph.addWeightedEdge('C', 'D', 2);
weightedGraph.addWeightedEdge('E', 'A', 1);
weightedGraph.addWeightedEdge('D', 'E', 1);
weightedGraph.addWeightedEdge('D', 'B', 1);
```

Output	
Observation	A directed weighted adjacency matrix captures both the travel direction and cost of every connection. The value at a given row and column shows the cost to travel from the row's node to the column's node, with zero denoting no direct link. Because paths flow one-way, the matrix does not need to mirror across its main diagonal.

A.2: Directed Weighted Adjacency Matrix

Code

```
class Graph{
private:
    std::map<char, std::vector<char>> adjList;

public:
    void addVertex(char v){
        adjList[v];
    }

    void addEdge(char u, char v){
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }

    void display(){
        for (const auto& [vertex, neighbors] : adjList){
            std::cout << vertex;

            for (char neighbor : neighbors)
                std::cout << " -> " << neighbor;

            std::cout << " -> NULL" << std::endl;
        }
    }
};
```

```
Graph g;

g.addVertex('A');
g.addVertex('B');
g.addVertex('C');
g.addVertex('D');
g.addVertex('E');

g.addEdge('A', 'B');
g.addEdge('A', 'C');
g.addEdge('A', 'D');
g.addEdge('B', 'E');
g.addEdge('C', 'D');
g.addEdge('D', 'E');

std::cout << "\nA3 - Adjacency List\n";

g.display();
```

Output	A3 - Adjacency List A -> B -> C -> D -> NULL B -> A -> E -> NULL C -> A -> D -> NULL D -> A -> C -> E -> NULL E -> B -> D -> NULL
Observation	In an undirected adjacency list, each node keeps a record of everything linked to it. Since pathways go back and forth equally, a single line between two points appears on both nodes' lists.

A.3: Undirected Adjacency List

Code Snippets

```
class DirectedGraph{
private:
    std::map<char, std::vector<char>> adjList;

public:
    void addVertex(char v){
        adjList[v];
    }

    void addEdge(char u, char v){
        adjList[u].push_back(v);
    }

    void display(){
        for (const auto& [vertex, neighbors] : adjList){
            std::cout << vertex;

            for (char neighbor : neighbors)
                std::cout << " -> " << neighbor;

            std::cout << " -> NULL" << std::endl;
        }
    }
};
```

```
DirectedGraph directedGraph;

directedGraph.addVertex('A');
directedGraph.addVertex('B');
directedGraph.addVertex('C');
directedGraph.addVertex('D');
directedGraph.addVertex('E');

directedGraph.addEdge('A', 'B');
directedGraph.addEdge('A', 'C');
directedGraph.addEdge('B', 'E');
directedGraph.addEdge('C', 'D');
directedGraph.addEdge('D', 'B');
directedGraph.addEdge('D', 'E');

std::cout << "\nA3 - Directed Graph Adjacency List\n";

directedGraph.display();
```

Output

```
A3 - Directed Graph Adjacency List
A -> B -> C -> NULL
B -> E -> NULL
C -> D -> NULL
D -> B -> E -> NULL
```

Observation

A directed adjacency list maps each node to the specific destinations it links to. Because connections flow in a single direction, a target node is only listed under the origin node that points toward it.

A.3: Directed Adjacency List

Code Snippets

```cpp
template <typename T>
void dfs(
    const GraphList<T>& G,
    size_t v,
    std::set<size_t>& visited,
    std::vector<size_t>& order
)
{
    visited.insert(v);
    order.push_back(v);

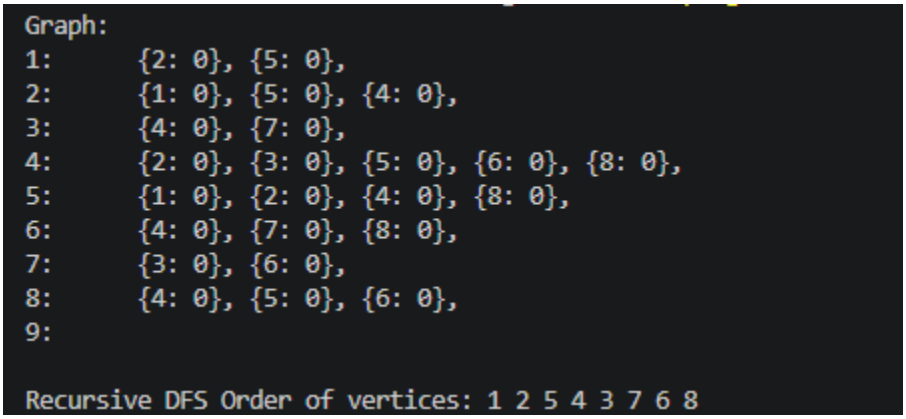
    for (auto e : G.outgoing_edges(v))
    {
        size_t u = e.dest;

        if (visited.find(u) == visited.end())
            dfs(G, u, visited, order);
    }
}

template <typename T>
std::vector<size_t> recursive_DFS(
    const GraphList<T>& G,
    size_t v
)
{
    std::set<size_t> visited;
    std::vector<size_t> order;

    dfs(G, v, visited, order);

    return order;
}
```

Output	
Observation	Recursive DFS traverses a graph by plunging as far as possible down a path, calling itself on each unvisited neighbor until it hits a dead end, where it then steps backward to try another route. A tracking set records each touched node, shielding the process from looping endlessly on cyclic pathways.

B.1: Depth First Search - Recursive

Code Snippets	

```cpp
template <typename T>
std::vector<size_t> iterative_DFS(const GraphList<T>& G, size_t v){
    std::stack<size_t> s;
    std::set<size_t> visited;
    std::vector<size_t> order;

    s.push(v);
    visited.insert(v);
    order.push_back(v);

    while (!s.empty()){
        size_t w = s.top();
        bool found = false;

        for (auto e : G.outgoing_edges(w)){
            size_t u = e.dest;

            if (visited.find(u) == visited.end()){
                s.push(u);
                visited.insert(u);
                order.push_back(u);

                found = true;
                break;
            }
        }

        if (!found){
            s.pop();
        }
    }

    return order;
}
```

```cpp
template <typename T>
GraphList<T> create_DFS_graph(){
    GraphList<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

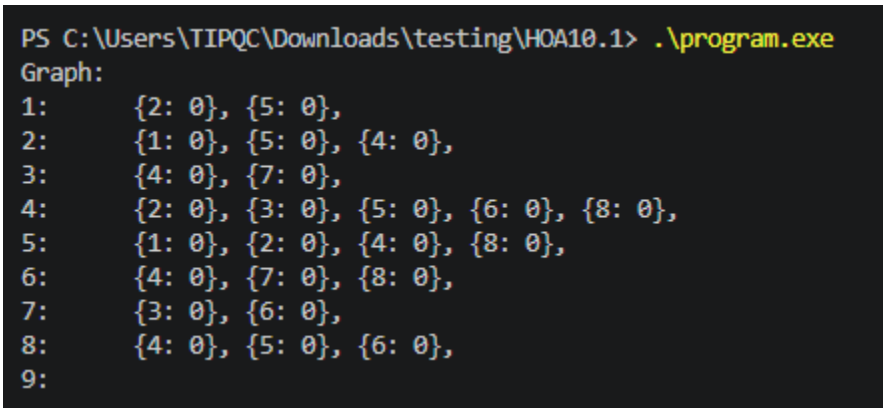
    for (auto& i : edges){
        for (auto& j : i.second){
            G.add_edge(
                Edge<T>{
                    i.first,
                    j.first,
                    j.second
                }
            );
        }
    }

    return G;
}
```

```cpp
int main()
{
    GraphList<unsigned> G = create_DFS_graph<unsigned>();

    std::cout << "Graph:\n";
    std::cout << G << std::endl;
```

```cpp
std::cout << "\nIterative DFS Order of vertices: ";
order = iterative_DFS(G, 1);
for (size_t v : order)
    std::cout << v << " ";
```

Output	
Observation	Iterative DFS mirrors the deep-first strategy of its recursive counterpart, swapping system call stacks for an explicit stack data structure. By operating under a Last-In, First-Out sequence, it prioritizes the most recently discovered nodes, while a dedicated log keeps track of evaluated elements to eliminate redundant passes.

B.1: Depth First Search - Iterative

Code Snippets	

```cpp
template <typename T>
std::vector<size_t> bfs(const GraphList<T>& G, size_t v){
    std::queue<size_t> q;
    std::set<size_t> visited;
    std::vector<size_t> order;

    q.push(v);
    visited.insert(v);
    order.push_back(v);

    while (!q.empty()){
        size_t w = q.front();
        q.pop();

        for (auto e : G.outgoing_edges(w)){
            size_t u = e.dest;

            if (visited.find(u) == visited.end()){
                visited.insert(u);
                q.push(u);
                order.push_back(u);
            }
        }
    }
    return order;
}
```

```cpp
template <typename T>
GraphList<T> create_BFS_graph(){
    GraphList<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto& i : edges){
        for (auto& j : i.second){
            G.add_edge(
                Edge<T>{i.first, j.first, j.second
                }
            );
        }
    }
    return G;
}
```

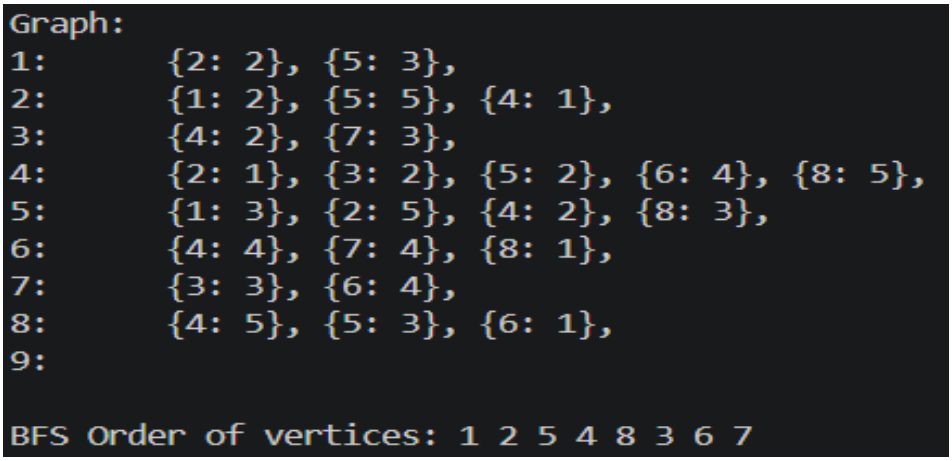
```cpp
int main(){
    GraphList<unsigned> G = create_BFS_graph<unsigned>();

    std::cout << "Graph:\n";
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices: ";
    std::vector<size_t> order = bfs(G, 1);

    for (size_t v : order)
        std::cout << v << " ";

    return 0;
}
```

Output	
Observation	Breadth-First Search traverses a network outward in expanding layers, processing all directly connected nodes prior to advancing deeper. It relies on a First-In, First-Out queue structure to order node evaluations, alongside a tracking mechanism that prevents duplicate visits.

B.2: Breadth First Search

2. SUPPLEMENTARY ACTIVITY

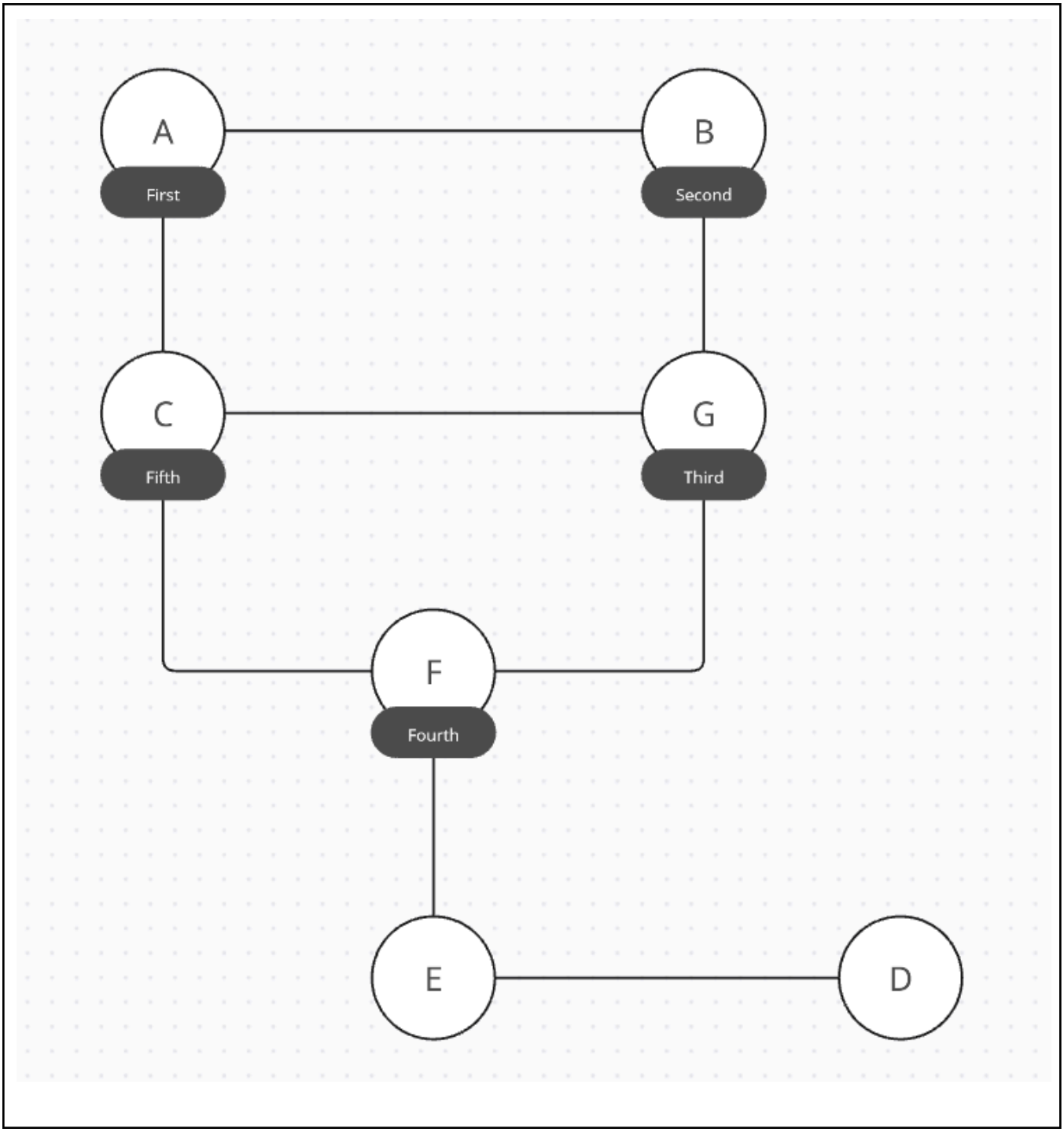
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

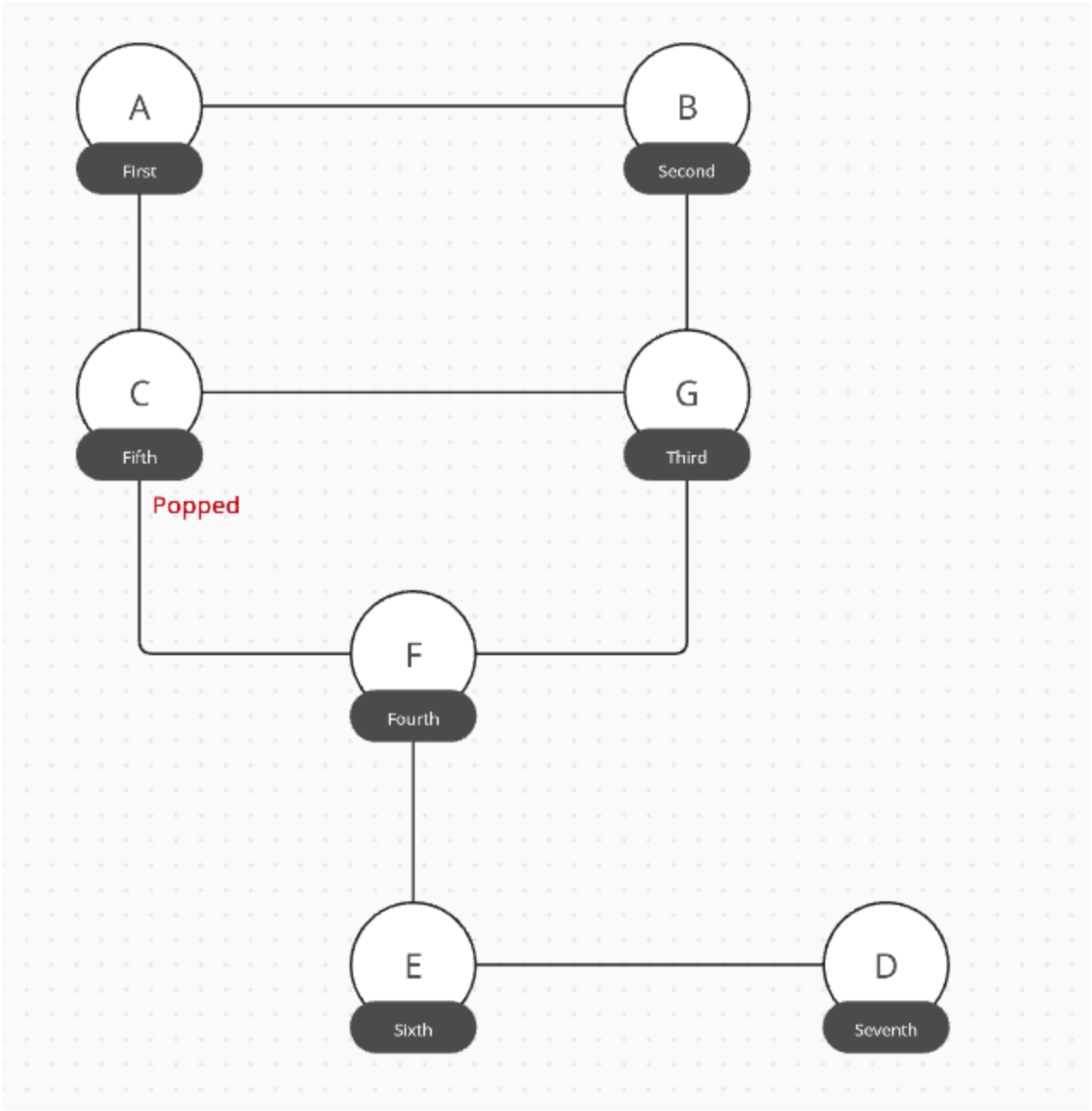
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- Depth-First Search is the most appropriate algorithm because it explores one path as deeply as possible before backtracking to the previous vertex. It then explores other unvisited vertices from that point, matching the described process of visiting locations and backtracking when a path is finished.

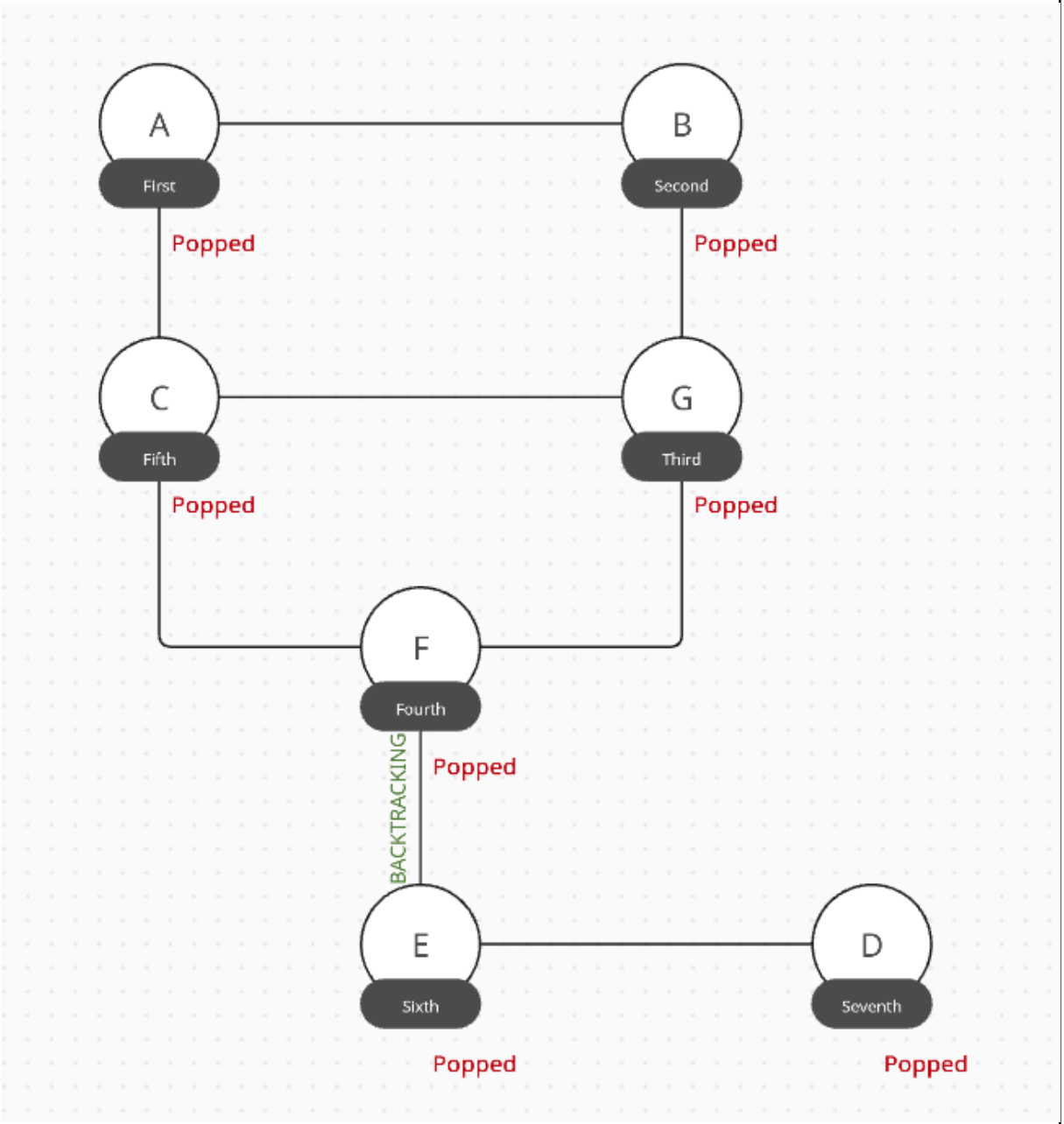
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

VISUALIZATION


A
First
B
Second
C
Fifth
G
Third
F
Fourth
E
D

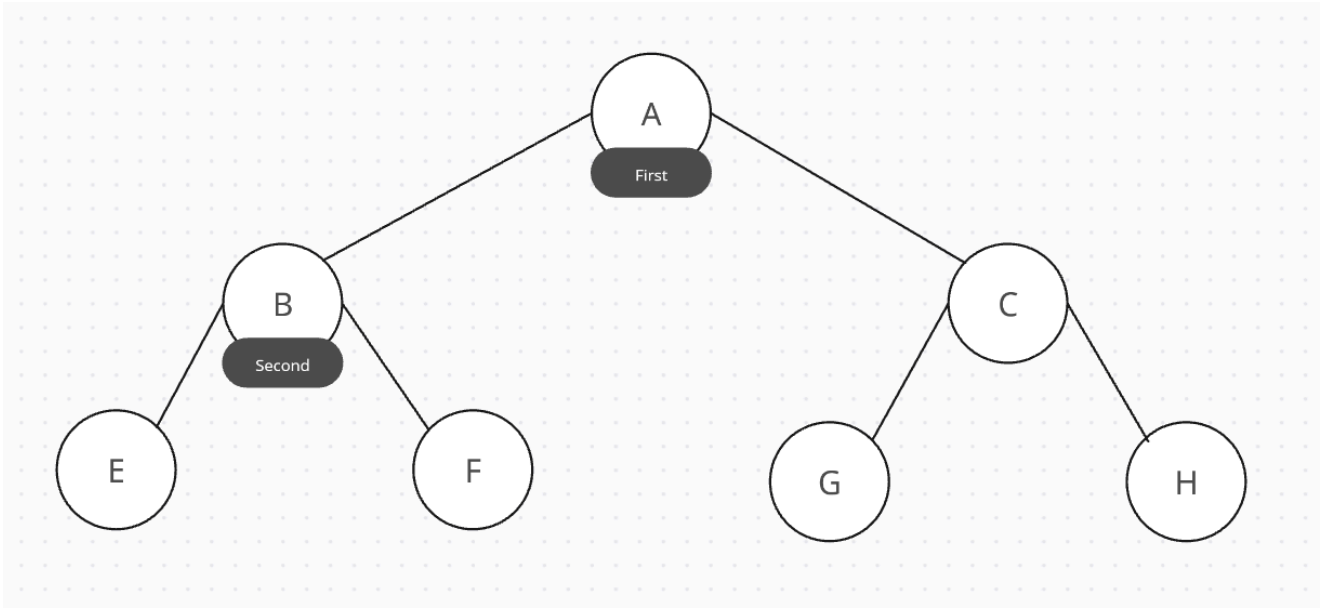


- At the fifth iteration, every neighbor of the node has been visited so now it pops or backtracks for other neighbors that haven't been visited yet.

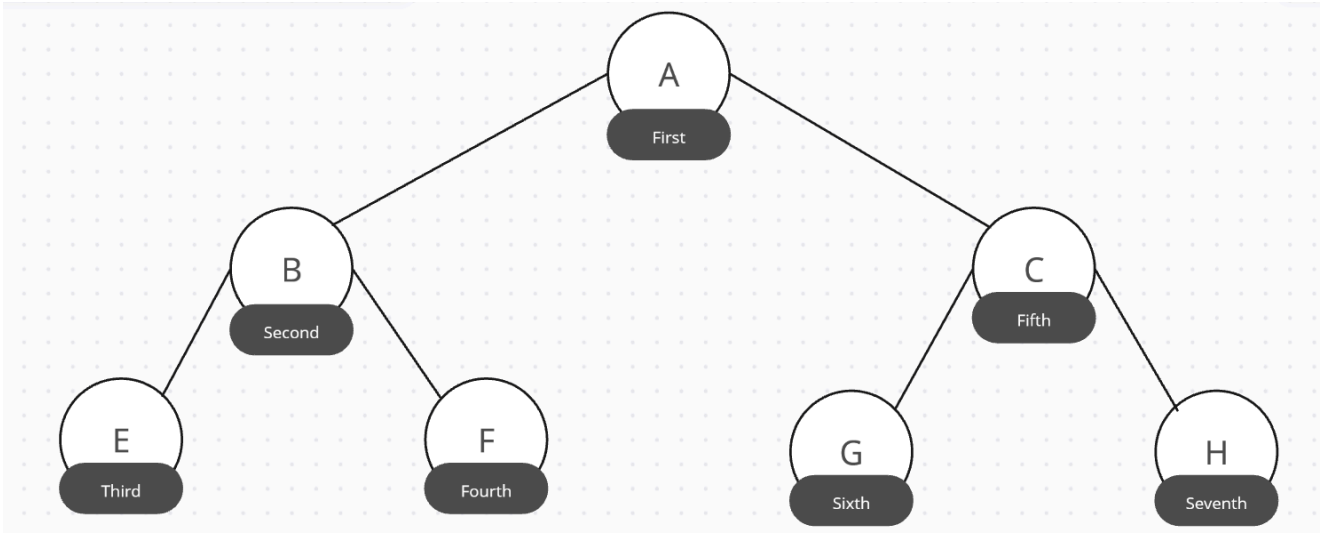


- At the seventh iteration, every node in the board has been visited, so it starts to back track and pop each node from the stack

DFS Graph Traversal Process



- The first and second nodes are visited, so they are popped from the stack. C, E, F, G, and H nodes remain waiting in the stack until they are visited.



- After visiting the children nodes of B, it then proceeds to C and the rest of the nodes, popping them as well

DFS Tree Traversal Process

graphDFS(start):	treeDFS(root):
Create an empty stack Create an empty visited set Push start onto the stack while stack is not empty: vertex = pop from stack if vertex is not visited: Mark vertex as visited Process vertex for each adjacent vertex: if it is not visited: Push it onto the stack	Create an empty stack Push root onto the stack while stack is not empty: node = pop from the stack if node is not visited: Process node Push right child onto the stack Push left child onto the stack

Pseudocode

ANALYSIS: These illustrate and showcase how Depth-First Search (DFS) works differently on graphs versus trees. Because graphs can contain cycles or multiple paths leading to the same node, graphDFS uses a visited set to track processed nodes, push unvisited neighbors onto a stack, and backtrack when stuck. In contrast, treeDFS traverses a hierarchical structure without cycles by systematically popping nodes from the stack and pushing right and left children to explore each branch top-down and left-to-right.

DFS Graph Code

```cpp
template <typename T>
std::vector<size_t> bfs(const GraphList<T>& G, size_t v){
    std::queue<size_t> q;
    std::set<size_t> visited;
    std::vector<size_t> order;

    q.push(v);
    visited.insert(v);
    order.push_back(v);

    while (!q.empty()){
        size_t w = q.front();
        q.pop();

        for (auto e : G.outgoing_edges(w)){
            size_t u = e.dest;

            if (visited.find(u) == visited.end()){
                visited.insert(u);
                q.push(u);
                order.push_back(u);
            }
        }
    }
    return order;
}
```

DFS Tree Code

```cpp
template <typename T>
struct TreeNode{
    T data;
    TreeNode<T>* left;
    TreeNode<T>* right;

    TreeNode(T value){
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

template <typename T>
std::vector<T> tree_DFS(TreeNode<T>* root){
    std::stack<TreeNode<T>*> s;
    std::vector<T> order;

    if (root == nullptr){
        return order;
    }

    s.push(root);

    while (!s.empty()){
        TreeNode<T>* node = s.top();
        s.pop();

        order.push_back(node->data);

        if (node->right != nullptr){
            s.push(node->right);
        }

        if (node->left != nullptr){
            s.push(node->left);
        }
    }
    return order;
}

#endif
```

```cpp
template <typename T>
GraphList<T> create_BFS_graph(){
    GraphList<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto& i : edges){
        for (auto& j : i.second){
            G.add_edge(
                Edge<T>{i.first, j.first, j.second
                }
            );
        }
    }
    return G;
}

#endif
```

OUTPUT

```
DFS Graph:
A: B C
B: G A
C: A G F
D: E
E: D F
F: C G E
G: B F C

DFS Order: A B G F C E D
Tree DFS Order: A B D E C F G
```

Code Implementation

3. In the performed code, what data structure is used to implement the Breadth First Search?
   - The standard library queue is used to implement Breadth-First Search. It follows the First In, First Out principle, where vertices are added to the back of the queue and processed from the front. This allows BFS to visit vertices level by level.

4. How many times can a node be visited in the BFS?
   - A node can be visited only once in the implemented BFS. A visited set keeps track of vertices that have already been visited, preventing them from being added to the queue and processed again. This also helps prevent repeated visits when multiple vertices are connected to the same node.

CONCLUSION

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code analysis, correction, and assistance, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was only used as a tool for assistance to improve upon my current knowledge and understanding. Apart from the stated usage above, all ideas, answers, and authorship remains to be my own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.